

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Laboratorio de Microcomputadoras

Práctica 07 Comunicación Serie Asíncrona UART

Grupo de laboratorio: 02

Profesor: Ing. Moisés Meléndez Reyes

Brigada: 04

Integrantes:

- Hernández Acosta Mauricio Gabriel 321121051
- Pérez Olvera Alexis Abraham 321150503
- Sierra García Mariana 320301238



Fecha de entrega: 25 de Octubre del 2026

Semestre: 2027-1

Índice

1. Objetivo **2**

2. Actividades **2**
2.1. Ejercicio 1 . . . 2
2.1.1. Pseudo Código . . . 3
2.1.2. Desarrollo . . . 3
2.1.3. Análisis de resultados . . . 4
2.2. Ejercicio 2 . . . 5
2.2.1. Desarrollo . . . 6
2.2.2. Análisis de resultados . . . 6
2.3. Ejercicio 3 . . . 7
2.3.1. Pseudo Código . . . 7
2.3.2. Desarrollo . . . 7
2.3.3. Análisis de resultados . . . 9
2.4. Ejercicio 4 . . . 10
2.4.1. Pseudo Código . . . 10
2.4.2. Desarrollo . . . 10
2.4.3. Análisis de resultados . . . 11
2.5. Ejercicio 5 . . . 13
2.5.1. Pseudo Código . . . 13
2.5.2. Desarrollo . . . 14
2.5.3. Análisis de resultados . . . 17
2.6. Ejercicio 6 . . . 18
2.6.1. Pseudo Código . . . 18
2.6.2. Desarrollo . . . 19
2.6.3. Análisis de resultados . . . 20
2.7. Ejercicio 7 . . . 21
2.7.1. Pseudo Código . . . 21
2.7.2. Desarrollo . . . 22
2.7.3. Análisis de resultados . . . 23

3. Conclusiones **24**
3.1. Hernandez Acosta Mauricio Gabriel . . . 24
3.2. Perez Olvera Alexis Abraham . . . 24
3.3. Sierra Garcia Mariana . . . 24

1. Objetivo

Aprender el funcionamiento de la comunicación serial en la modalidad asíncrona para la transferencia de información entre diferentes dispositivos por medios alámbricos e inalámbricos.

2. Actividades

Realizar las actividades solicitadas, en ellas se realizará la comunicación serie asíncrona, será empleada el formato REPL usando la interfaz por default; así como las siguientes interfaces.

2.1. Ejercicio 1

Escribir el siguiente programa:

```
import select
import sys
import time
import machine

poll_obj = select.poll()
poll_obj.register(sys.stdin, 1)
sys.stdout.write("Esperando recepción de datos \n")
print("Tecla un carácter y luego <enter>")
while True:
    if poll_obj.poll(0):
        ch = sys.stdin.read(1)
        sys.stdout.write("Dato recibido \n")
        print("Hola UNAM")
        time.sleep(0.1)
```

a Comentar el código.

b Indicar que hace.

c Ejecutar.

d Ingresar datos desde la consola de Thonny.

e Explicar la diferencia entre:

   a `sys.stdout.write(.Esperando recepción de datos \n")`

   b `print("Teclea un caracter y luego <enter>")`

2.1.1. Pseudo Código

Algoritmo 1: Lectura de datos desde entrada estándar

Inicializar sistema de lectura no bloqueante;
Registrar entrada estándar;
Mostrar .Esperando recepción de datos";
Mostrar "Tecla un carácter y luego <enter>";
Mientras verdadero hacer;
    Verificar si hay datos disponibles en la entrada;
    if *hay datos disponibles* **then**
        Leer un carácter;
        Mostrar "Dato recibido";
        Mostrar "Hola UNAM";
        Esperar 0.1 segundos;
    end

2.1.2. Desarrollo

```python
import select
import sys
import time
import machine

# Crea un objeto 'poll' para realizar un seguimiento de eventos de entrada
poll_obj = select.poll()

# Registra la entrada estándar (teclado/consola) en el objeto poll
# El valor '1' indica que queremos monitorear el evento POLLIN (datos listos para leer)
poll_obj.register(sys.stdin, 1)

# Envía un mensaje a la consola indicando que el programa está listo
sys.stdout.write("Esperando recepción de datos \n")
print("Teclea un carácter y luego <enter>")

while True:
    # Verifica si hay datos disponibles en los objetos registrados (timeout de 0ms)
    # Si poll(0) devuelve una lista con contenido, significa que hay datos esperando
    if poll_obj.poll(0):
        # Lee un solo carácter de la entrada estándar
        ch = sys.stdin.read(1)

        # Confirma la recepción del dato
        sys.stdout.write("Dato recibido \n")

        # Imprime un saludo
        print("Hola UNAM")

    time.sleep(0.1)
```



Ejecución Ejercicio 1

2.1.3. Análisis de resultados

El programa funcionó correctamente al implementar una lectura no bloqueante de la entrada estándar mediante el uso de `poll()`, lo que permitió detectar cuando el usuario ingresaba datos sin detener la ejecución del ciclo principal. Se comprobó que cada vez que se presionaba una tecla seguida de *enter*, el sistema respondía mostrando los mensajes "Dato recibido" y "Hola UNAM". Además, el uso de un retardo ayudó a evitar lecturas repetidas excesivas, manteniendo un comportamiento estable en la comunicación serial dentro del entorno REPL.

2.2. Ejercicio 2

Para este ejercicio, realizar los pasos siguientes:

a. Ubicar el COM de conexión de la Raspberry Pi.

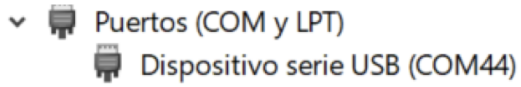


Figura 7-4. Información del Administrador de Dispositivos

b. Abrir la terminal de su preferencia.

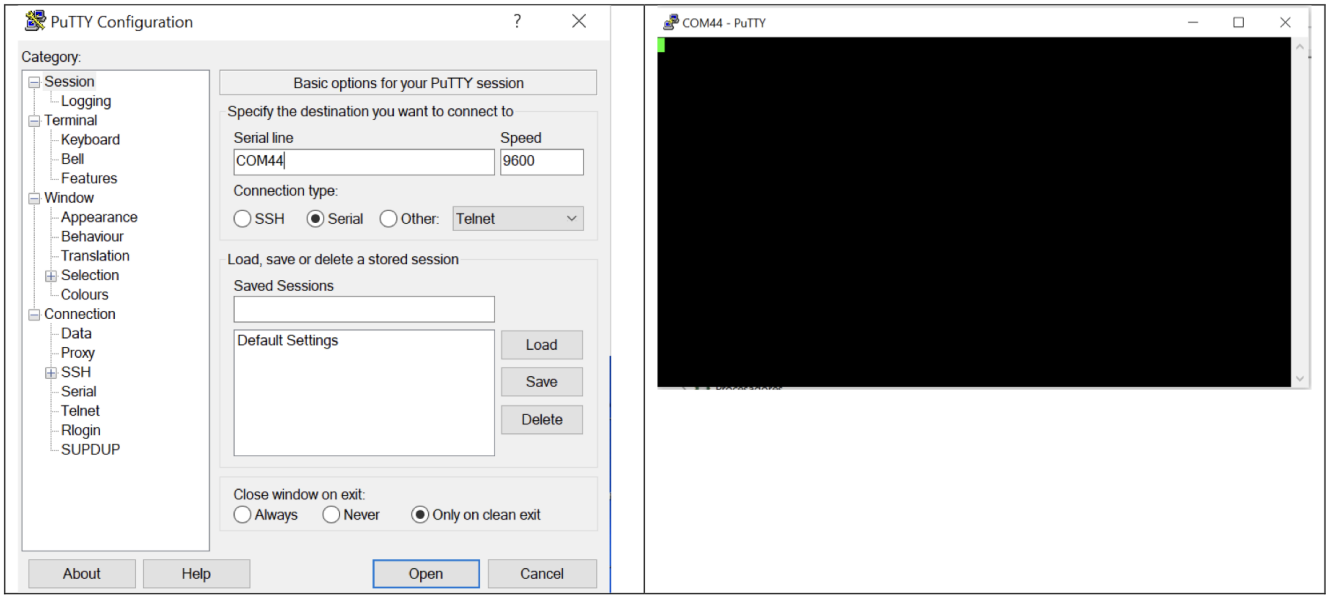


Figura 7-5. Configuración de Putty

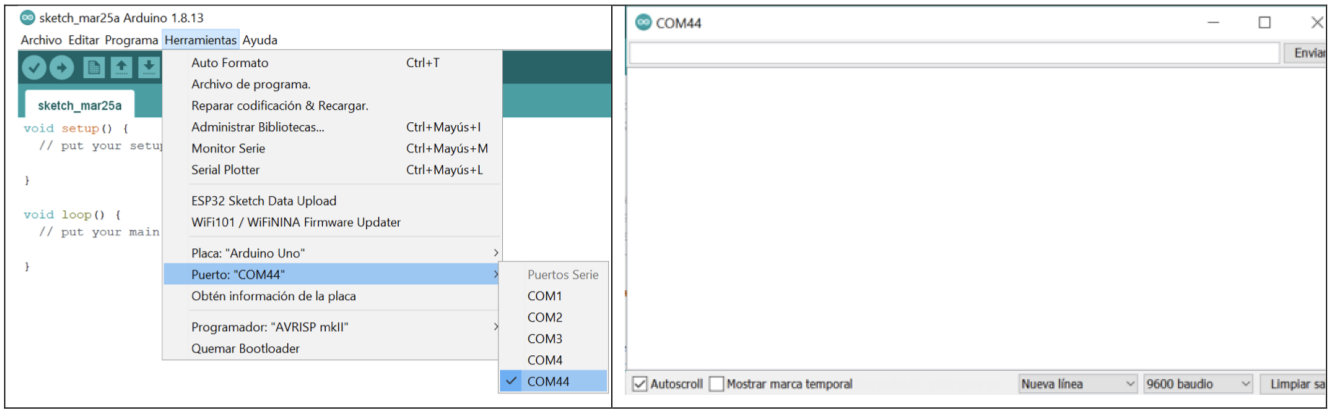


Figura 7-6. Configuración en monitor de Arduino

c. Explicar si existe alguna diferencia en la ejecución y cuál sería el motivo.

2.2.1. Desarrollo

Sí existe una diferencia en la ejecución al utilizar distintas interfaces como Thonny, PuTTY o el monitor serial de Arduino. La principal diferencia radica en la forma en que cada terminal gestiona la comunicación serial y el envío de caracteres. Algunas terminales envían automáticamente caracteres de fin de línea (como `\n` o `\r\n`), mientras que otras requieren configuraciones específicas como baud rate o control de flujo. Esto puede provocar variaciones en la recepción de datos, aunque la lógica del programa permanece correcta.



Ejecución Ejercicio 2

2.2.2. Análisis de resultados

Durante la ejecución en diferentes terminales, se observó que el programa mantiene su funcionalidad de recepción y respuesta de datos; sin embargo, la interacción puede variar ligeramente según la herramienta utilizada. En todos los casos, el sistema detectó correctamente la entrada de caracteres y respondió de forma adecuada, confirmando la correcta implementación de la comunicación serial asíncrona. Las diferencias observadas se deben principalmente a la configuración de cada terminal y no afectan el funcionamiento general del programa.

2.3. Ejercicio 3

Utilizando el formato de la actividad 1, realizar un programa que reciba comandos a través del puerto serie y ejecute las acciones indicadas.

Dato recibido	Acción
'0'	GPIO25 = OFF
'1'	GPIO25 = ON

2.3.1. Pseudo Código

Algoritmo 2: Control de LED mediante entrada serial

```
Configurar GPIO25 como salida (LED);
Inicializar sistema de lectura no bloqueante;
Registrar entrada estándar;
Mostrar "Sistema de control iniciado";
Mostrar "Comandos: 1 para encender, 0 para apagar";
Mientras verdadero hacer;
    Verificar si hay datos disponibles en la entrada;
    if hay datos then
        Leer un carácter;
        if carácter = '1' then
            Encender LED (GPIO25 = 1);
            Mostrar "GPIO25 encendido";
        else
            if carácter = '0' then
                Apagar LED (GPIO25 = 0);
                Mostrar "GPIO25 apagado";
            end
        end
    end
    Esperar 0.1 segundos;
```

2.3.2. Desarrollo

```python
import select
import sys
import time
import machine

# Configuración del GPIO 25 como salida (Output)
led = machine.Pin(25, machine.Pin.OUT)

# Configuración del objeto poll para monitorear la entrada serial
poll_obj = select.poll()
poll_obj.register(sys.stdin, 1)

sys.stdout.write("Sistema de control iniciado\n")
print("Comandos: '1' para Encender, '0' para Apagar")

while True:
```

```
    # Verificamos si hay datos en el búfer de entrada
    if poll_obj.poll(0):
        # Leemos el carácter recibido
        ch = sys.stdin.read(1)

        # Lógica de control según el dato recibido
        if ch == '1':
            led.value(1)  # GPIO25 = ON
            sys.stdout.write("Acción: GPIO25 encendido (ON)\n")

        elif ch == '0':
            led.value(0)  # GPIO25 = OFF
            sys.stdout.write("Acción: GPIO25 apagado (OFF)\n")

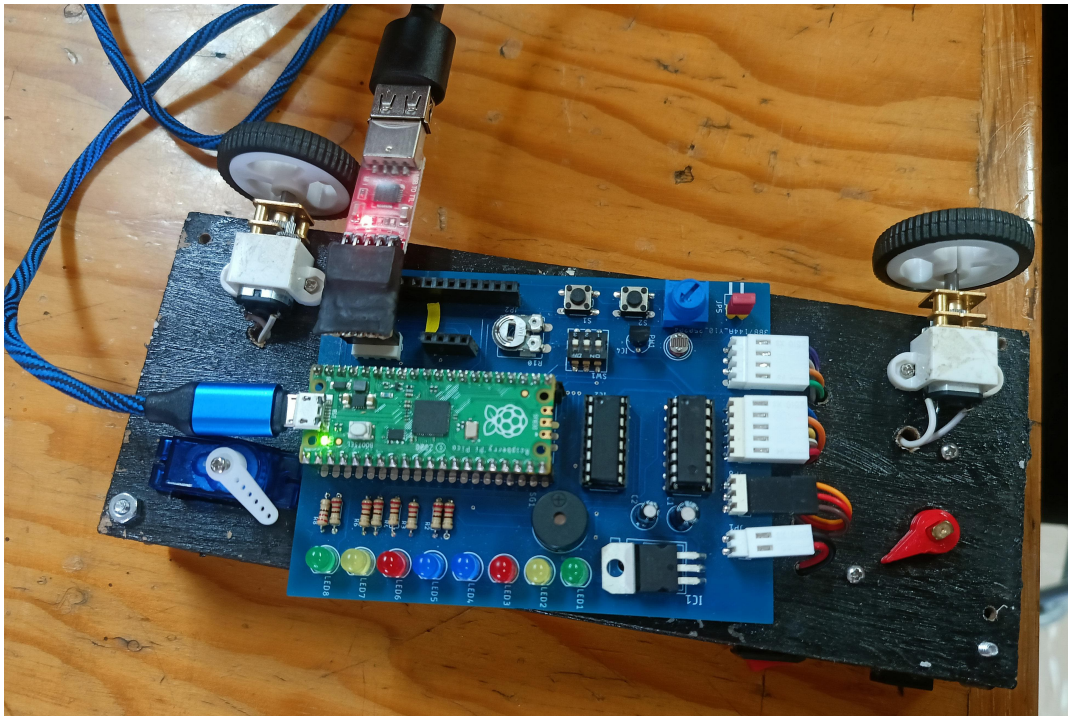
        # Nota: Los caracteres como '\n' (Enter) son ignorados
        # al no coincidir con '1' o '0'.

    # Pausa para estabilidad del sistema
    time.sleep(0.1)
```

```
Shell
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Sistema de control iniciado
  Comandos: '1' para Encender, '0' para Apagar
  1
  Acción: GPIO25 encendido (ON)
  0
  Acción: GPIO25 apagado (OFF)
```

Ejecución Ejercicio 3



Raspberry Pi Ejercicio 3

2.3.3. Análisis de resultados

El sistema respondió correctamente a los comandos enviados desde la terminal, permitiendo controlar el estado del LED mediante caracteres específicos. Se comprobó que la lectura no bloqueante evitó interrupciones en la ejecución del programa, mientras que la lógica condicional filtró adecuadamente los datos válidos, ignorando caracteres como saltos de línea. La interacción fue inmediata y consistente, demostrando un control eficiente de salidas digitales mediante comunicación serial.

2.4. Ejercicio 4

Realizar el siguiente procedimiento:

a. Escribir el siguiente programa.

b. Comentar el código e indicar que hace.

c. Conectar el módulo CP2102.

d. Ubicar el puerto asignado.

e. Comprobar el funcionamiento

```
from machine import Pin, UART
import time

uart = UART(0, baudrate=9600, tx=Pin(16), rx=Pin(17))
uart.init(bits=8, parity=None, stop=1)
led = Pin(25, Pin.OUT)
uart.write('Inicia Comunicacion Serie')
while True:
    if uart.any() > 0:
        data = uart.read()
        uart.write(data)
        led.toggle()
        time.sleep(1)
```

2.4.1. Pseudo Código

Algoritmo 3: Comunicación UART con control de LED

Configurar UART en puerto 0 con baudrate 9600;
Configurar pines TX y RX;
Configurar LED en GPIO25 como salida;
Enviar mensaje “Inicia Comunicación Serie”;
Mientras verdadero hacer;
  if *hay datos disponibles en UART* **then**
    Leer datos recibidos;
    Enviar de regreso los mismos datos (eco);
    Cambiar estado del LED (toggle);
    Esperar 1 segundo;
  end

2.4.2. Desarrollo

```
from machine import Pin, UART
import time

# Inicializa la UART0 con velocidad de 9600 baudios
# TX en GPIO16 y RX en GPIO17
uart = UART(0, baudrate=9600, tx=Pin(16), rx=Pin(17))

# Configuración adicional de la UART: 8 bits, sin paridad, 1 bit de parada
uart.init(bits=8, parity=None, stop=1)

```

```python
# Configura el LED en GPIO25 como salida
led = Pin(25, Pin.OUT)

# Envía un mensaje inicial por la UART
uart.write('Inicia Comunicacion Serie')

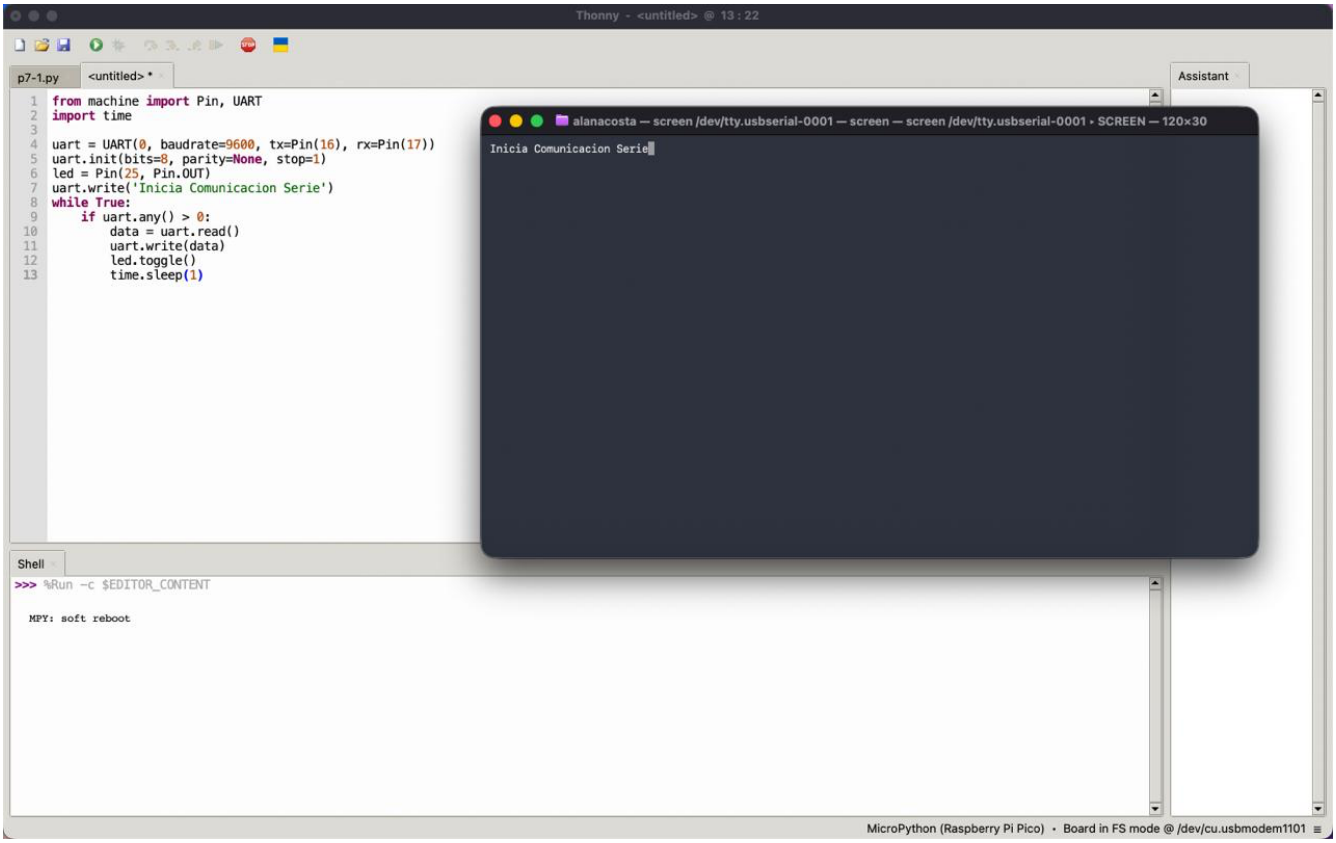
while True:
    # Verifica si hay datos disponibles en el buffer de recepción
    if uart.any() > 0:

        # Lee los datos recibidos
        data = uart.read()

        # Envía de regreso los mismos datos (eco)
        uart.write(data)

        # Cambia el estado del LED (encendido/apagado)
        led.toggle()

        # Espera 1 segundo para evitar cambios muy rápidos
        time.sleep(1)
```



Ejecución Ejercicio 4

2.4.3. Análisis de resultados

Durante la implementación de la comunicación UART con el módulo CP2102, se observó un funcionamiento correcto en la transmisión y recepción de datos. El programa logró realizar un eco de

los datos recibidos, confirmando la integridad de la comunicación. Además, el cambio de estado del LED permitió visualizar físicamente la recepción de información. El retardo implementado evitó cambios demasiado rápidos, mejorando la estabilidad del sistema.

2.5. Ejercicio 5

Siguiendo el procedimiento de la actividad No. 4; realizar un programa que ejecute las acciones indicadas.

Dato UART	Acción
'1'	Mostrar el voltaje del Potenciómetro
'2'	Mostrar el resultado de la conversión AD de la foto-resistencia en hexadecimal.
'3'	Mostrar la temperatura del sensor TMP36 en °C, °F y °K.
'4'	Reproducir la nota DO en el zumbador
'5'	Prender y apagar 5 veces GPIO0 al GPIO7
'6'	Corrimiento a la derecha de los 8 leds GPIO0 – GPIO7
'7'	Corrimiento a la izquierda de los 8 leds GPIO0 – GPIO7

2.5.1. Pseudo Código

Algoritmo 4: Sistema multitarea con UART

```
Configurar UART, ADC (pot, LDR, temperatura);
Configurar LEDs (GPIO 0–7) y buzzer;
Mostrar "Sistema iniciado";
Mientras verdadero hacer;
    if hay comando en UART then
        Leer comando;
        if cmd = '1' then
            Leer potenciómetro y enviar voltaje;
        else
            if cmd = '2' then
                Leer LDR y enviar valor en hexadecimal;
            else
                if cmd = '3' then
                    Leer temperatura y enviar en C, F, K;
                else
                    if cmd = '4' then
                        Activar buzzer (nota DO);
                    else
                        if cmd = '5' then
                            Parpadear LEDs;
                        else
                            if cmd = '6' then
                                Corrimiento de LEDs a la derecha;
                            else
                                if cmd = '7' then
                                    Corrimiento de LEDs a la izquierda;
                                end
                            end
                        end
                    end
                end
            end
        end
    end
end
```

2.5.2. Desarrollo

```
from machine import Pin, UART, ADC, PWM
import time

# 1. Configuración de UART (igual que Actividad 4)
uart = UART(0, baudrate=9600, tx=Pin(16), rx=Pin(17))
uart.init(bits=8, parity=None, stop=1)

# 2. Configuración de Entradas Analógicas (ADC)
pot = ADC(26)      # Potenciómetro
ldr = ADC(27)      # Foto-resistencia
tmp36 = ADC(28)    # Sensor de temperatura

# 3. Configuración de Salidas (LEDs GPIO 0 al 7)
led_pins = [Pin(i, Pin.OUT) for i in range(8)]

# 4. Configuración del Zumbador (PWM)
buzzer = PWM(Pin(15))
buzzer.duty_u16(0) # Iniciamos en silencio

def apagar_leds():
    for led in led_pins:
        led.value(0)

uart.write('Sistema Multitarea Iniciado. Ingrese comando (1-7):\r\n')

while True:
    if uart.any() > 0:
        cmd = uart.read().decode('utf-8').strip()

        # --- ACCIÓN 1: Voltaje del Potenciómetro ---
        if cmd == '1':
            reading = pot.read_u16()
            voltage = reading * (3.3 / 65535)
            uart.write('Voltaje Pot: {:.2f}V\r\n'.format(voltage))

        # --- ACCIÓN 2: LDR en Hexadecimal ---
        elif cmd == '2':
            val_ldr = ldr.read_u16()
            uart.write('LDR (Hex): {}\r\n'.format(hex(val_ldr)))

        # --- ACCIÓN 3: Temperatura TMP36 ---
        elif cmd == '3':
            # Conversión de lectura a grados Celsius
            m_voltios = tmp36.read_u16() * (3300 / 65535)
            celsius = (m_voltios - 500) / 10
            fahrenheit = (celsius * 9/5) + 32
            kelvin = celsius + 273.15

            msg = 'Temp: {:.2f}C, {:.2f}F, {:.2f}K\r\n'.format(celsius, fahrenheit, kelvi
            uart.write(msg)

        # --- ACCIÓN 4: Nota DO en Zumbador ---
        elif cmd == '4':
            uart.write('Reproduciendo DO (261Hz)...\r\n')
            buzzer.freq(261)   # Frecuencia de la nota DO (C4)
```

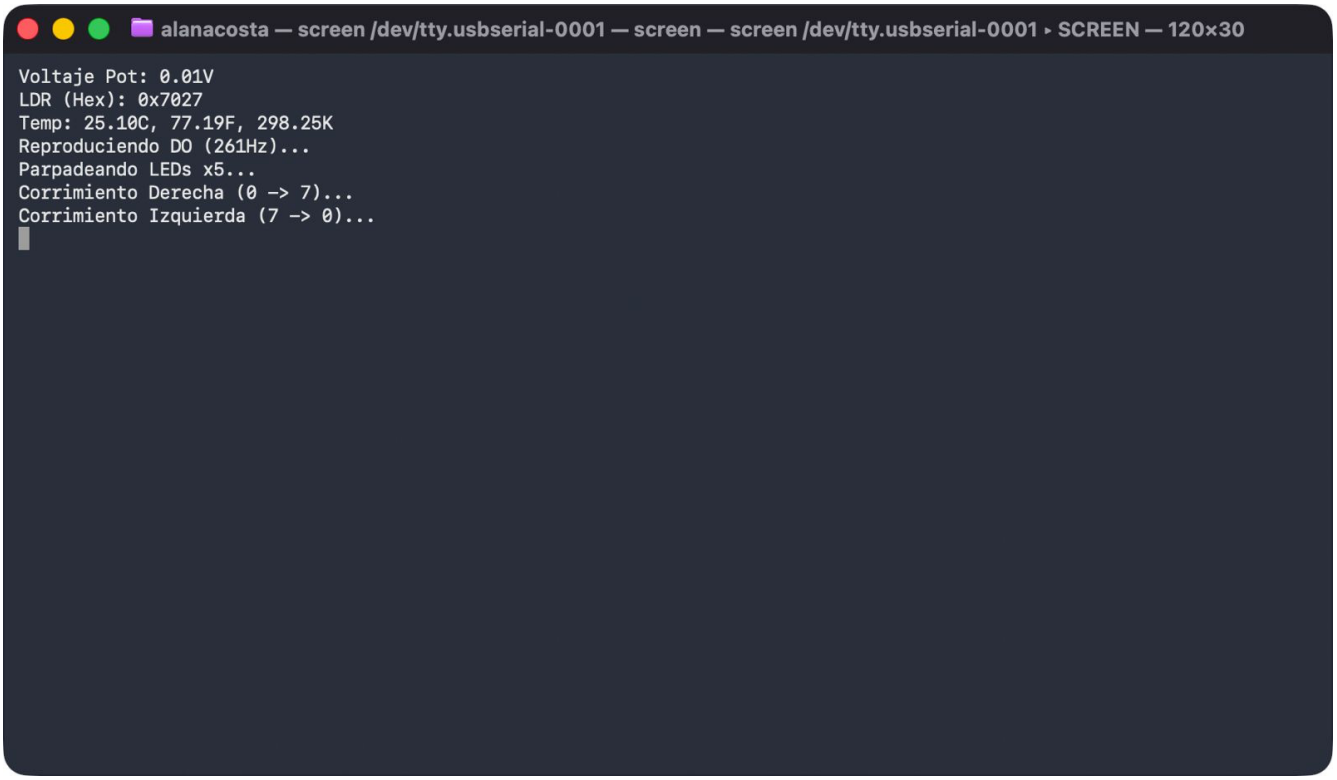
```
            buzzer.duty_u16(32768) # 50% de ciclo de trabajo
            time.sleep(0.5)
            buzzer.duty_u16(0)

        # --- ACCIÓN 5: Parpadeo GPIO 0-7 (5 veces) ---
        elif cmd == '5':
            uart.write('Parpadeando LEDs x5...\r\n')
            for _ in range(5):
                for led in led_pins: led.value(1)
                time.sleep(0.2)
                for led in led_pins: led.value(0)
                time.sleep(0.2)

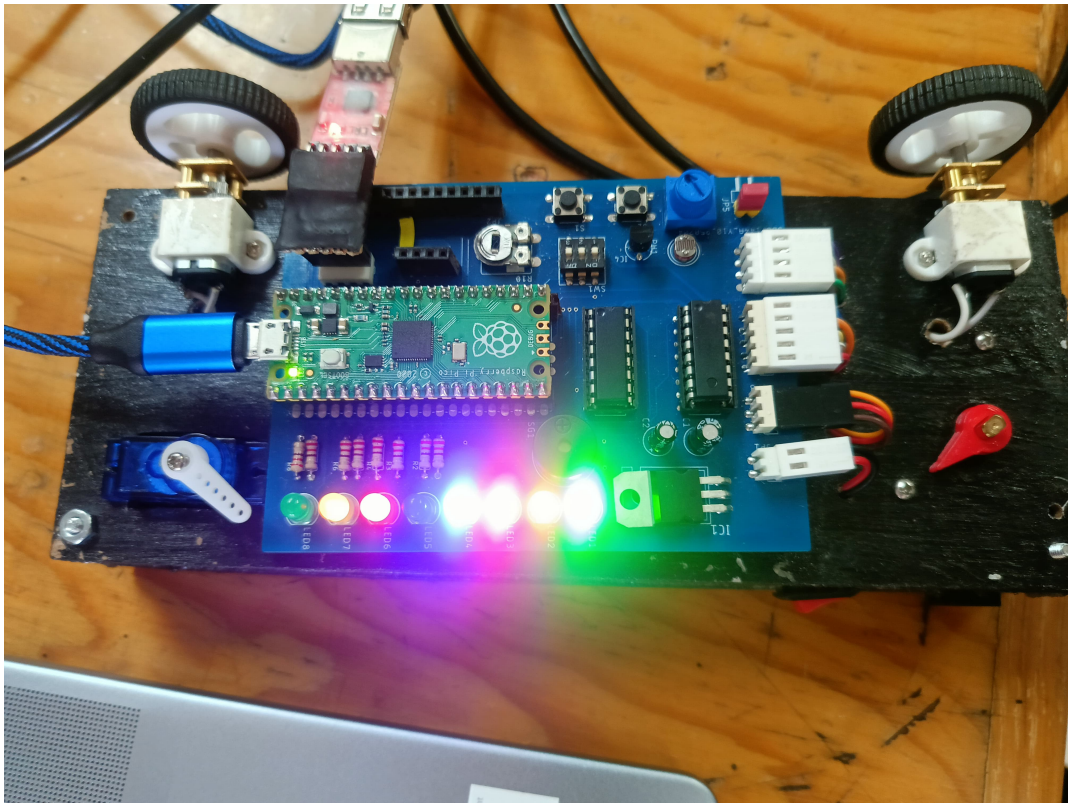
        # --- ACCIÓN 6: Corrimiento a la derecha ---
        elif cmd == '6':
            uart.write('Corrimiento Derecha (0 -> 7)...\r\n')
            apagar_leds()
            for led in led_pins:
                led.value(1)
                time.sleep(0.1)
                led.value(0)

        # --- ACCIÓN 7: Corrimiento a la izquierda ---
        elif cmd == '7':
            uart.write('Corrimiento Izquierda (7 -> 0)...\r\n')
            apagar_leds()
            for led in reversed(led_pins):
                led.value(1)
                time.sleep(0.1)
                led.value(0)

    time.sleep(0.1)
```



Ejecución Ejercicio 5



Raspberry Pi Ejercicio 5

2.5.3. Análisis de resultados

El sistema multitarea mostró un comportamiento adecuado al ejecutar distintas acciones dependiendo del comando recibido. Se verificó la correcta lectura de sensores analógicos (potenciómetro, LDR y temperatura), así como la conversión de datos a diferentes unidades. Las salidas digitales (LEDs) y el buzzer respondieron correctamente, evidenciando la integración de múltiples periféricos en un solo programa. La estructura condicional permitió una ejecución clara y organizada de cada función.

2.6. Ejercicio 6

Conectando el módulo bluetooth, realizar un programa que ejecute las acciones indicadas en la tabla 7-4.

Dato recibido	Acción
'A'	GPIO0 = ON
'T'	GPIO1 = ON
'D'	GPIO2 =ON
'I'	GPIO3 =ON
S	GPIO0:GPIO3 = OFF

2.6.1. Pseudo Código

Algoritmo 5: Control de motores por UART (Bluetooth)

```
Configurar UART (9600 baudios);
Configurar pines de motores (Puente H);
Definir función detener motores;
Definir función para controlar dirección de motores;
Mostrar "Sistema listo";
Mientras verdadero hacer;
    if hay datos en UART then
        Leer comando y convertir a mayúsculas;
        if cmd = 'S' then
            Detener ambos motores;
        else
            if cmd = 'A' then
                Ambos motores en sentido horario;
            else
                if cmd = 'T' then
                    Ambos motores en sentido antihorario;
                else
                    if cmd = 'D' then
                        Motor 1 horario, Motor 2 antihorario;
                    else
                        if cmd = 'I' then
                            Motor 1 antihorario, Motor 2 horario;
                        end
                    end
                end
            end
        end
    end
    Esperar 0.1 segundos;
```

2.6.2. Desarrollo

```
from machine import Pin, UART
import time

# 1. Configuración de UART (Bluetooth en GP16 y GP17)
uart = UART(0, baudrate=9600, tx=Pin(16), rx=Pin(17))

# 2. Configuración de Pines para Motores (Puente H)
# Motor 1
m1_a = Pin(2, Pin.OUT)
m1_b = Pin(3, Pin.OUT)
# Motor 2
m2_a = Pin(4, Pin.OUT)
m2_b = Pin(5, Pin.OUT)

# Función para detener ambos motores
def stop():
    m1_a.value(0); m1_b.value(0)
    m2_a.value(0); m2_b.value(0)

# Función para establecer giro (1=Horario, -1=Anti-horario, 0=Paro)
def set_motors(m1_dir, m2_dir):
    # Lógica Motor 1
    if m1_dir == 1:    # Horario
        m1_a.value(1); m1_b.value(0)
    elif m1_dir == -1: # Anti-horario
        m1_a.value(0); m1_b.value(1)
    else:              # Paro
        m1_a.value(0); m1_b.value(0)

    # Lógica Motor 2
    if m2_dir == 1:    # Horario
        m2_a.value(1); m2_b.value(0)
    elif m2_dir == -1: # Anti-horario
        m2_a.value(0); m2_b.value(1)
    else:              # Paro
        m2_a.value(0); m2_b.value(0)

uart.write("Control de Motores Listo (A, T, D, I, S)\r\n")

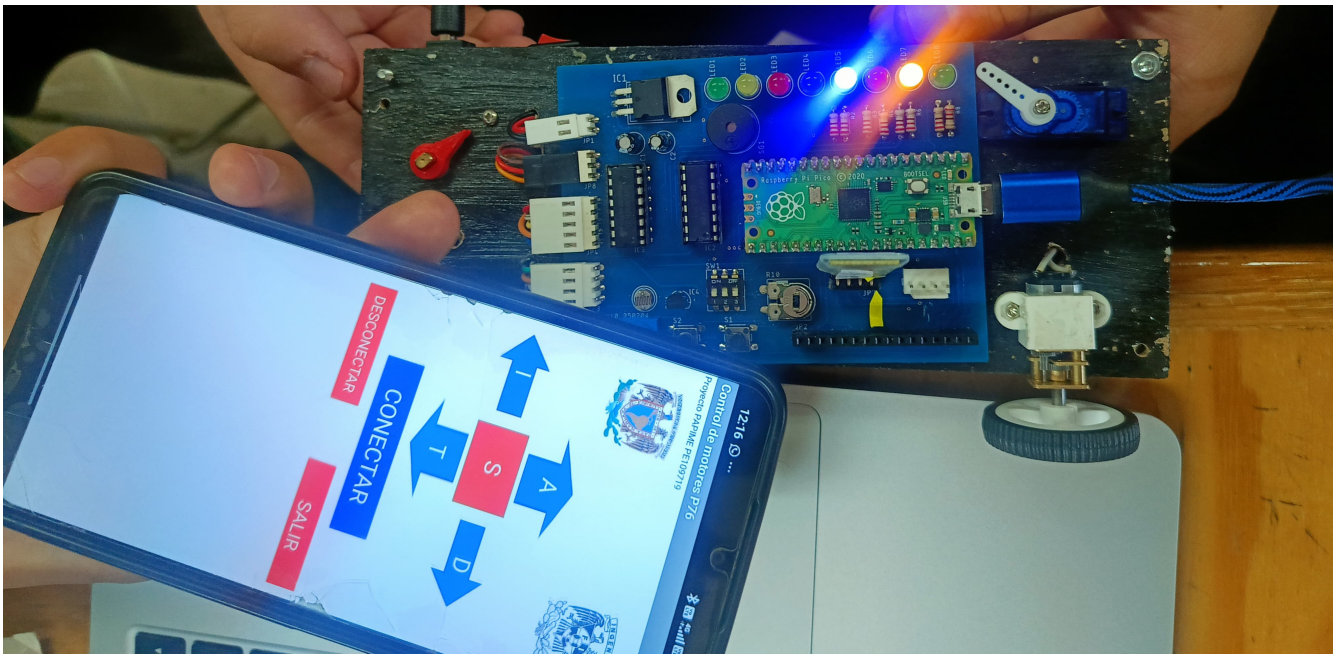
while True:
    if uart.any() > 0:
        # Leemos el comando y lo pasamos a mayúsculas
        comando = uart.read().decode('utf-8').strip().upper()

        if comando == 'S':
            stop()
            uart.write("ACCION: PARO - PARO\r\n")

        elif comando == 'A':
            set_motors(1, 1)
            uart.write("ACCION: HORARIO - HORARIO\r\n")

        elif comando == 'T':
            set_motors(-1, -1)
            uart.write("ACCION: ANTI-HORARIO - ANTI-HORARIO\r\n")
```

```
56
57          elif comando == 'D':
58              set_motors(1, -1)
59              uart.write("ACCION: HORARIO - ANTI-HORARIO\r\n")
60
61          elif comando == 'I':
62              set_motors(-1, 1)
63              uart.write("ACCION: ANTI-HORARIO - HORARIO\r\n")
64
65      time.sleep(0.1)
```



Raspberry Pi Ejercicio 6

2.6.3. Análisis de resultados

El control de motores mediante comunicación Bluetooth fue exitoso, permitiendo ejecutar diferentes movimientos según los comandos enviados. Se comprobó que la lógica implementada en el puente H permitió invertir el sentido de giro de los motores correctamente. La respuesta del sistema fue inmediata, lo que demuestra una comunicación eficiente. Asimismo, la normalización de los comandos (uso de mayúsculas) evitó errores de interpretación.

2.7. Ejercicio 7

Realizar un programa que controle el funcionamiento de los motores de Corriente Directa de la practica 5, por medio de la comunicación serie empleando el módulo Bluetooth.

Comando Puerto Serie	ACCION	
	MOTOR 1	MOTOR 2
"S"	PARO	PARO
"A"	HORARIO	HORARIO
"T"	ANTI-HORARIO	ANTI-HORARIO
"D"	HORARIO	ANTI-HORARIO
"I"	ANTI-HORARIO	HORARIO

2.7.1. Pseudo Código

Algoritmo 6: Control de motores con PWM por UART

```
Configurar UART (Bluetooth);
Configurar pines de dirección de ambos motores;
Configurar PWM para control de velocidad;
Definir velocidad máxima;
Definir función para controlar motores (dirección y velocidad);
Mostrar "Sistema listo";
Mientras verdadero hacer;
    if hay datos en UART then
        Leer comando y convertir a mayúsculas;
        if cmd = 'S' then
            Detener ambos motores;
        else
            if cmd = 'A' then
                Ambos motores avanzan (máxima velocidad);
            else
                if cmd = 'T' then
                    Ambos motores retroceden;
                else
                    if cmd = 'D' then
                        Giro a la derecha;
                    else
                        if cmd = 'I' then
                            Giro a la izquierda;
                        end
                    end
                end
            end
        end
    end
    Esperar 0.1 segundos;
```

2.7.2. Desarrollo

```
from machine import Pin, UART, PWM
import time

# 1. Configuración de UART (Bluetooth en GP16 y GP17)
uart = UART(0, baudrate=9600, tx=Pin(16), rx=Pin(17))

# 2. Configuración Motor 1 (Según imagen)
m1_dir1 = Pin(0, Pin.OUT)  # DIR1_M1
m1_dir2 = Pin(1, Pin.OUT)  # DIR2_M1
m1_en = PWM(Pin(20))       # EN_M1 (Habilitación/Velocidad)
m1_en.freq(1000)           # Frecuencia de 1kHz para el motor

# 3. Configuración Motor 2 (Según imagen)
m2_dir1 = Pin(2, Pin.OUT)  # DIR1_M2
m2_dir2 = Pin(3, Pin.OUT)  # DIR2_M2
m2_en = PWM(Pin(19))       # EN_M2 (Habilitación/Velocidad)
m2_en.freq(1000)

def set_motors(d1, d2, d3, d4, speed_m1, speed_m2):
    # Motor 1
    m1_dir1.value(d1)
    m1_dir2.value(d2)
    m1_en.duty_u16(speed_m1)
    # Motor 2
    m2_dir1.value(d3)
    m2_dir2.value(d4)
    m2_en.duty_u16(speed_m2)

# Velocidad máxima (ciclo de trabajo al 100%)
MAX_SPEED = 65535

uart.write("Control de Motores UNAM - Listo\r\n")

while True:
    if uart.any() > 0:
        comando = uart.read().decode('utf-8').strip().upper()

        if comando == 'S':  # PARO - PARO
            set_motors(0, 0, 0, 0, 0, 0)
            uart.write("ACCION: PARO\r\n")

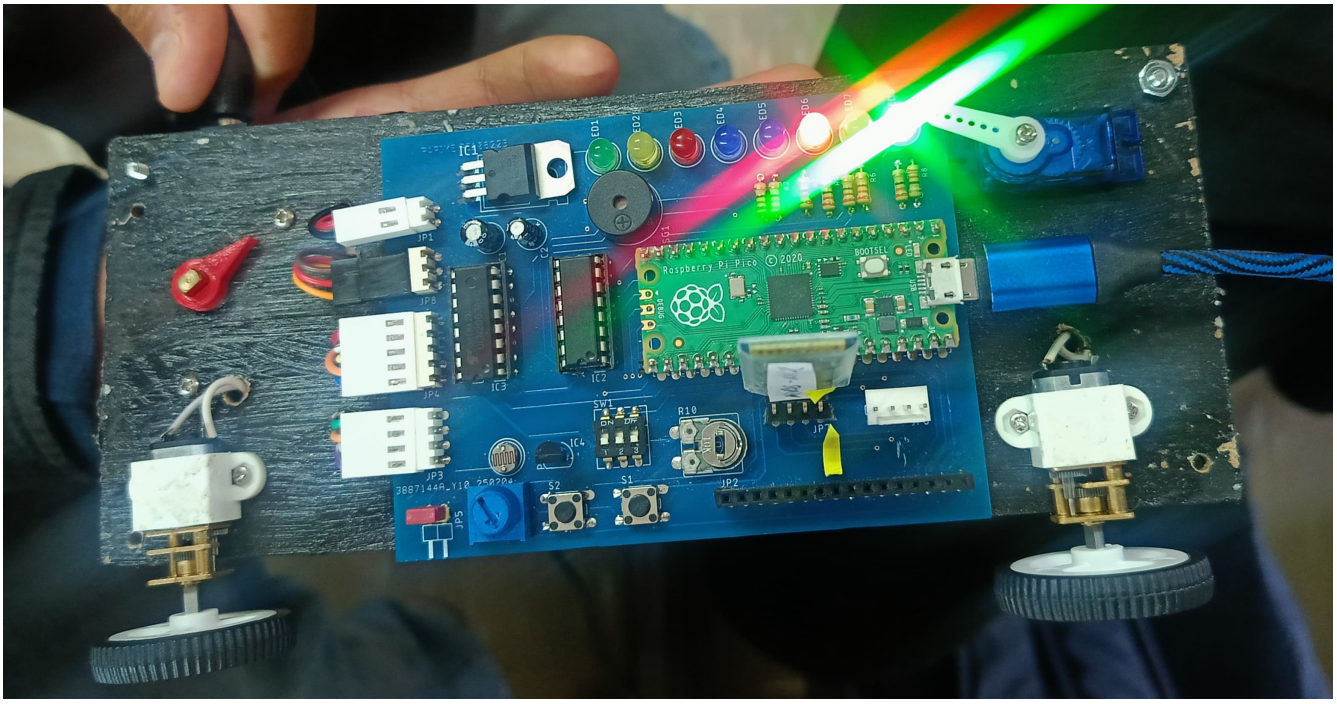
        elif comando == 'A':  # HORARIO - HORARIO
            set_motors(1, 0, 1, 0, MAX_SPEED, MAX_SPEED)
            uart.write("ACCION: AVANZAR (H-H)\r\n")

        elif comando == 'T':  # ANTI-HORARIO - ANTI-HORARIO
            set_motors(0, 1, 0, 1, MAX_SPEED, MAX_SPEED)
            uart.write("ACCION: RETROCEDER (AH-AH)\r\n")

        elif comando == 'D':  # HORARIO - ANTI-HORARIO
            set_motors(1, 0, 0, 1, MAX_SPEED, MAX_SPEED)
            uart.write("ACCION: DERECHA (H-AH)\r\n")

        elif comando == 'I':  # ANTI-HORARIO - HORARIO
            set_motors(0, 1, 1, 0, MAX_SPEED, MAX_SPEED)
```

```
56              uart.write("ACCION: IZQUIERDA (AH-H)\r\n")
57
58      time.sleep(0.1)
```



Raspberry Pi Ejercicio 7

2.7.3. Análisis de resultados

La incorporación de PWM permitió un control más avanzado de los motores, no solo en dirección sino también en velocidad. Se observó que los motores respondieron adecuadamente a los comandos de avance, retroceso y giros, manteniendo una velocidad constante al utilizar el ciclo de trabajo máximo. La integración con Bluetooth funcionó de manera estable, evidenciando una mejora respecto al ejercicio anterior al añadir control de potencia.

3. Conclusiones

El desarrollo de esta práctica se llevó a cabo gracias al tutorial de Raspberry Pi de la facultad [1] y a las clases del profesor Moisés [2].

3.1. Hernandez Acosta Mauricio Gabriel

Trabajar con la comunicación serie asíncrona en esta práctica me permitió entender de forma muy clara cómo se transmiten datos entre dos puntos sin necesidad de compartir una línea de reloj común. Lo que más me llamó la atención fue la diferencia entre hacer una lectura que detiene todo el programa y utilizar mecanismos de sondeo no bloqueantes como `poll()` o `uart.any()`; ver que el microcontrolador puede seguir atendiendo otras tareas mientras espera la llegada de un carácter cambia por completo la manera de diseñar el código. Ajustar parámetros como los baudios, la paridad y los bits de parada me ayudó a valorar la precisión temporal requerida en los protocolos UART para evitar pérdidas de información.

3.2. Perez Olvera Alexis Abraham

Esta práctica fue muy dinámica porque pasamos de enviar caracteres simples por la consola a construir un sistema completo de comandos para gobernar periféricos en tiempo real. Me pareció sumamente interesante cómo un simple byte recibido por el puerto serie puede desencadenar lecturas analógicas del potenciómetro, calcular temperaturas o emitir frecuencias específicas en el zumbador mediante PWM. Además, vincular el módulo Bluetooth a las terminales UART de la Raspberry Pi Pico y manipular las funciones desde una aplicación móvil demostró lo directo y versátil que resulta migrar de una conexión alámbrica a un control remoto inalámbrico sin tener que reescribir la lógica del programa.

3.3. Sierra Garcia Mariana

Lo que más disfruté de la práctica fue ver la aplicación física del control serie en los motores de corriente directa. Al principio parecía complejo coordinar las señales del puente H junto con la recepción de datos inalámbricos, pero al estructurar las funciones de giro y velocidad con PWM todo fluyó de forma natural. Comprobar que podíamos detener, arrancar y cambiar el sentido de giro de los motores enviando comandos desde el celular por Bluetooth me dio una perspectiva muy real de cómo se controlan los vehículos y robots en la industria. En definitiva, la sesión me dejó aprendizajes muy valiosos sobre comunicación asíncrona que complementan lo visto en prácticas anteriores.

Referencias

[1] R. Anaya García et al., *Manual de Aplicaciones Raspberry Pi*, Manual de recursos y aplicaciones para plataforma Raspberry Pi, Universidad Nacional Autónoma de México, Facultad de Ingeniería, Laboratorio de Microcomputadoras, Ciudad Universitaria Coyoacán, México, 2026. dirección: https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf

[2] M. Meléndez Reyes, *Notas de clase: Práctica 07 - Comunicación Serie Asíncrona UART*, Clase presencial en laboratorio de microprocesadores, Facultad de Ingeniería, UNAM, Material proporcionado por el profesor durante la sesión de laboratorio, abr. de 2026.